README.md

🔍 AI Universal Recovery System

A sophisticated **Lost & Found platform** powered by **AI Vision** and **Vector Search**. It uses a microservices architecture to match lost items with found items using semantic image understanding (CLIP models) rather than just text keywords.

![Project Screenshot](https://via.placeholder.com/800x400?text=Dashboard+Screenshot+Here)

(Tip: Replace this link with a real screenshot of your Dashboard)

🚀 Key Features

- ** 🔮 AI-Powered Matching:** Uses OpenAI's **CLIP model** to generate vector embeddings for images. Matches "Lost" items to "Found" items based on visual similarity, not just text.

- ** ⚡ Real-time Alerts:** Instant email notifications when a high-confidence match is found.

- ** 💬 Secure Chat:** Private, real-time messaging between the owner and finder (Socket.IO).

- ** 🛡️ Secure Auth:** Full authentication flow with JWT, Email Verification, and Forgot Password mechanisms.

📐 Architecture

- **Core Backend:** NestJS (Node.js) with TypeORM & PostgreSQL.

- **Vision Service:** Python (FastAPI) running PyTorch & YOLO/CLIP.

- **Vector DB:** Milvus (Zilliz) for high-speed similarity search.

- **Frontend:** React + Vite + TailwindCSS.

- **Storage:** MinIO (S3 compatible) for object storage.

🛠️ Tech Stack

| Component | Technology |

|----------|----------|

| **Backend** | NestJS, TypeScript, TypeORM |

| **AI / ML** | Python, FastAPI, PyTorch, CLIP, YOLO |

| **Database** | PostgreSQL, Redis, Milvus (Vector DB) |

| **Frontend** | React, TailwindCSS, Lucide Icons |

| **DevOps** | Docker, Docker Compose |

🔧 Getting Started

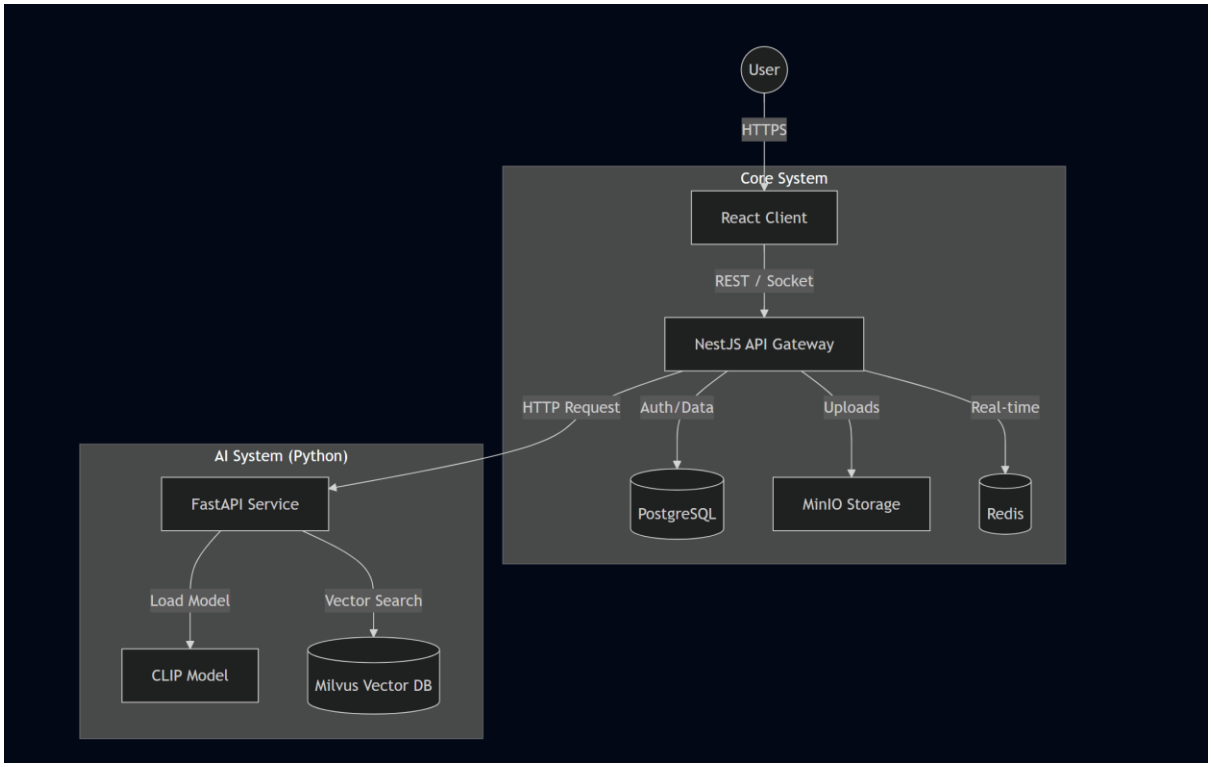
Prerequisites

- Node.js & npm
- Python 3.9+
- Docker Desktop (for DBs)

Installation

1. **Clone the Repo**

   ```bash

   git clone [https://github.com/yourusername/universal-recovery-system.git](https://github.com/yourusername/universal-recovery-system.git)

graph TD

    User((User)) -->|HTTPS| Frontend[React Client]

    subgraph "Core System"

        Frontend -->|REST / Socket| API[NestJS API Gateway]

        API -->|Auth/Data| DB[(PostgreSQL)]

        API -->|Uploads| MinIO[MinIO Storage]

        API -->|Real-time| Redis[(Redis)]

    end

    subgraph "AI System (Python)"

        API -->|HTTP Request| Vision[FastAPI Service]

        Vision -->|Load Model| CLIP[CLIP Model]

        Vision -->|Vector Search| Milvus[(Milvus Vector DB)]

    end
   ```



apps/core-backend/src/items/entities/item.entity.ts

```
import { Entity, PrimaryGeneratedColumn, Column, ManyToOne, CreateDateColumn } from 'typeorm';

import { User } from '../../users/entities/user.entity';


@Entity()

export class Item {

  @PrimaryGeneratedColumn('uuid')

  id: string;


  @Column()

  description: string;


  @Column()

  imageUrl: string; // URL to MinIO


  @Column("simple-array", { nullable: true })

  tags: string[]; // ["backpack", "red"] from YOLO
```

```
  @Column("float", { array: true, nullable: true })
  vector: number[]; // [0.12, -0.5...] from CLIP

  @Column({ default: false })
  isLost: boolean; // true = lost, false = found

  @ManyToOne(() => User, (user) => user.id)
  user: User; // Who posted this?

  @CreateDateColumn()
  createdAt: Date;
}
```

apps/core-backend/src/users/entities/user.entity.ts

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn, UpdateDateColumn } from
'typeorm';

@Entity()
export class User {
  @PrimaryGeneratedColumn('uuid') // Generates unique IDs like 'a1b2-c3d4...'
  id: string;

  @Column({ unique: true })
  email: string;

  @Column()
  passwordHash: string; // We never store raw passwords!

  @Column({ nullable: true })
  fullName: string;
```

```
  @Column({ default: false })
  isVerified: boolean; // For KYC verification later

  // 🆕 NEW FIELDS FOR RESET
  @Column({ nullable: true, type: 'varchar' })
  resetPasswordToken: string | null;

  @Column({ nullable: true, type: 'timestamp' })
  resetPasswordExpires: Date | null;

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;
}
```

apps/core-backend/src/chat/conversation.entity.ts

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn, UpdateDateColumn,
OneToMany, ManyToOne } from 'typeorm';
import { Message } from './message.entity';
import { User } from '../users/entities/user.entity';

@Entity()
export class Conversation {
    @PrimaryGeneratedColumn('uuid')
    id: string;

    @Column()
    itemId: string;
```

```
  // We explicitly link users to make looking up "My Configured Chats" easy
  @ManyToOne(() => User)
  finder: User;

  @ManyToOne(() => User)
  receiver: User;

  @OneToMany(() => Message, (message) => message.conversation)
  messages: Message[];

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;
}
```

apps/core-backend/src/chat/message.entity.ts

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn, ManyToOne } from
'typeorm';
import { User } from '../users/entities/user.entity';
import { Conversation } from './conversation.entity';

@Entity()
export class Message {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column()
  content: string;
```

```
  // New Flag: If true, the frontend will show " 🚫 Message deleted"
  @Column({ default: false })
  isDeleted: boolean;

  @ManyToOne(() => Conversation, (conversation) => conversation.messages, { nullable: true,
onDelete: 'CASCADE' })
  conversation: Conversation;

  @ManyToOne(() => User)
  sender: User;

  @CreateDateColumn()
  createdAt: Date;
}
```